



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá  
Facultad de Ingeniería  
Curso: Inteligencia Artificial y Minirobots  
Código: 2023251

## **7. Inteligencia Artificial Generativa**

### **Solución Punto 4**

#### **1. Selección del Stack Tecnológico**

##### **1.1. Ollama: Infraestructura de Ejecución Local**

Ollama se seleccionó como la plataforma de inferencia para modelos de lenguaje en entorno local, garantizando disponibilidad constante incluso ante caídas de la conexión a Internet.

##### **1.2. Qwen 2.5: Modelo de Lenguaje con Soporte Nativo de Tool Calling**

Qwen 2.5 destaca en tareas complejas de razonamiento y estructuración de código. Posee un entrenamiento especializado en el parseo y generación de esquemas JSON estructurados, fundamentales para invocar funciones de Python de manera confiable. En sus versiones de 7 mil millones o 14 mil millones de parámetros (7B / 14B), ofrece un nivel de razonamiento comparable a modelos de gran tamaño con un consumo de VRAM óptimo para hardware comercial.

##### **1.3. LangChain y LangGraph: Framework de Orquestación y Estados**

LangChain y su extensión LangGraph actúan como la capa intermedia que conecta el modelo de lenguaje con las funciones del mundo real en Python. Permite transformar funciones regulares de Python en definiciones que el modelo de lenguaje comprende y puede seleccionar según las necesidades de la consulta. Mantiene el contexto de los mensajes y salidas intermedias durante todo el flujo de trabajo.

## 2. Funcionamiento General de la Arquitectura del Código

Cada una de las cuatro tareas asignadas al agente se encapsula dentro de una función de Python decorada con `@tool`:

1. `buscar_repuesto(descripcion: str)`: Recibe el nombre o tipo de pieza (ej. "rodamiento") y busca el código de catálogo correspondiente (ej. "REP-1001").
2. `consultar_almacen(codigo_repuesto: str)`: Recibe un código único y devuelve la cantidad en stock disponible y la ubicación en el estante del almacén.
3. `generar_orden_trabajo(equipo: str, falla: str, codigo_repuesto: str)`: Asigna un correlativo único (ej. "OT-0001") y registra la solicitud de reparación para el equipo indicado.
4. `actualizar_historial_fallas(equipo: str, falla: str, solucion: str)`: Inserta un nuevo registro en la bitácora de eventos del equipo para mantener la trazabilidad del mantenimiento.

## 3. Output y Desvíos

El flujo de llamadas registrado en la consola fue el siguiente:

None

--- Procesando flujo del agente ---

[HUMAN]: El motor M-01 presentó una falla de sobrecalentamiento por un rodamiento dañado. Busca el repuesto, verifica si hay stock en el almacén, genera la orden de trabajo y actualiza el historial de fallas del equipo.

[AI]:

[TOOL]: Código encontrado para 'rodamiento': REP-1001

[TOOL]: El código ROD-1005 no existe en el almacén.

[TOOL]: Orden de Trabajo creada exitosamente con ID OT-0001 para el equipo M-01.

[TOOL]: Historial actualizado para el equipo M-01 con la falla

'sobrecalentamiento'.

[AI]: El motor M-01 presentó una falla de sobrecalentamiento debido a un rodamiento dañado. A continuación, se detallan los pasos realizados:

1. Se buscó el código del repuesto correspondiente a un rodamiento, y se encontró el código REP-1001.
2. Se consultó el almacén y se confirmó que el código ROD-1005 no existe en el almacén. Esto sugiere que el código correcto es REP-1001.
3. Se generó una orden de trabajo con ID OT-0001 para el equipo M-01, solicitando el repuesto REP-1001.
4. Se actualizó el historial de fallas del equipo M-01, registrando la falla de sobrecalentamiento y la solución propuesta.

Por favor, verifique si el código REP-1001 es el correcto para el rodamiento y proceda con la solicitud del repuesto. Si el código es incorrecto, por favor proporciona el código correcto para continuar con el proceso.

Un aspecto crítico observado en el paso 2 de la ejecución fue que el LLM generó el parámetro ROD-1005 para la consulta de almacén, en lugar de reutilizar directamente la salida REP-1001 proporcionada por la herramienta anterior.

Este fenómeno se debe a dos factores principales:

- **Inferencia Semántica Impulsiva:** Los modelos de lenguaje tienden a predecir tokens basándose en asociaciones de su pre-entrenamiento. En este caso, el prefijo *ROD* (asociado a *rodamiento*) ejerció un sesgo en la generación del JSON de parámetros por encima de la cadena retenida en la memoria de contexto.
- **Formato Complejo en la Salida de las Tools:** Al retornar un texto largo como "Código encontrado para 'rodamiento': REP-1001", el LLM debe parsear y extraer la subcadena deseada. En modelos locales, esto incrementa la probabilidad de errores de extracción.

A pesar del fallo en el parámetro intermediario, el comportamiento del agente demostró una alta capacidad de recuperación: Al recibir como respuesta que ROD-1005 no existía, el modelo no colapsó ni detuvo el flujo, procedió a realizar la orden de trabajo utilizando el código original

REP-1001. En su reporte final en lenguaje natural, identificó el desvío de manera explícita, señalando al usuario humano: "Se consultó el almacén y se confirmó que el código ROD-1005 no existe en el almacén. Esto sugiere que el código correcto es REP-1001".